

Aufgabenblatt: Arrays

Aufgabe 1: Farben-Array

Beschreibung: In Arrays hat jedes Element eine feste Position, den Index, der bei 0 beginnt. Eine `for`-Schleife mit Laufvariable erlaubt es, in jedem Durchlauf sowohl auf den Index als auch auf den Wert zuzugreifen und beides gleichzeitig auszugeben.

Aufgabenstellung: Lege folgendes Array direkt im Code an:

```
String[] farben = {"Rot", "Blau", "Gruen"};
```

Gib jedes Element mit seiner Indexposition aus.

```
Index 0: Rot  
Index 1: Blau  
Index 2: Gruen
```

Aufgabe 2: Summe eines Arrays

Beschreibung: Schleifen und Arrays ergaenzen sich natuerlich: Eine Akkumulator-Variable sammelt in jedem Durchlauf einen Wert, waehrend die Schleife das Array Element fuer Element abarbeitet. Das Ergebnis steht nach dem letzten Durchlauf fest.

Aufgabenstellung: Lege folgendes Array direkt im Code an:

```
int[] zahlen = {2, 4, 6, 8, 10};
```

Berechne die Summe aller Elemente mit einer Schleife und gib das Ergebnis aus.

```
Summe: 30
```

Aufgabe 3: Namen durchsuchen

Beschreibung: Beim Suchen in einem String-Array wird jedes Element nacheinander mit dem gesuchten Wert verglichen. In Java muss dieser Vergleich mit `.equals()` erfolgen: Der `==`-Operator prüft nur, ob zwei Variablen auf dasselbe Objekt im Speicher zeigen, nicht ob ihre Inhalte identisch sind. Gross- und Kleinschreibung werden dabei unterschieden.

Aufgabenstellung: Lege folgendes Array direkt im Code an:

```
String[] namen = {"Anna", "Max", "Tom", "Lisa"};
```

Lies einen Namen mit dem Scanner ein. Suche ihn im Array und gib aus, ob er gefunden wurde oder nicht. Gross- und Kleinschreibung werden dabei beachtet.

```
Bitte gib einen Namen ein: Max
Max wurde im Array gefunden.
```

```
Bitte gib einen Namen ein: Klaus
Name nicht gefunden.
```

Aufgabe 4: Notendurchschnitt mit Bewertung

Beschreibung: Eine Schleife kann ein Array nicht nur lesen, sondern auch befüllen: Jede Eingabe landet an der nächsten freien Position. Nach dem Befüllen steht das gesamte Array für Berechnungen und Auswertungen zur Verfügung.

Aufgabenstellung: Lese 5 Noten als Kommazahlen per Scanner ein und speichere sie in einem `double`-Array. Berechne den Durchschnitt und gib ihn auf zwei Nachkommastellen aus. Gib anschliessend eine Bewertung nach folgender Tabelle aus:

Durchschnitt	Bewertung
Unter 2,0	Sehr gut
Unter 3,0	Gut
Unter 4,0	Befriedigend
Ab 4,0	Verbesserungsbedarf

```
Note 1: 1,5
Note 2: 2,0
Note 3: 3,0
Note 4: 2,0
Note 5: 2,5
Der Durchschnitt ist: 2,20
```

[Bonus] Matrix transponieren

Beschreibung: Beim Transponieren einer Matrix werden Zeilen zu Spalten: Was vorher an Position Zeile 0, Spalte 2 stand, steht hinterher an Position Zeile 2, Spalte 0. Mathematisch gilt: $\text{transponiert}[\text{spalte}][\text{zeile}] = \text{original}[\text{zeile}][\text{spalte}]$. Verschachtelte Schleifen fuehren diese Vertauschung systematisch fuer alle Positionen durch.

Aufgabenstellung: Lege folgendes 3x3-Array direkt im Code an:

```
int[][] zahlen = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};
```

Gib das Original und anschliessend die transponierte Form jeweils zeilenweise aus. Die Transposition kann direkt in der Ausgabe umgesetzt werden.

Vorher:

```
1 2 3  
4 5 6  
7 8 9
```

Nachher:

```
1 4 7  
2 5 8  
3 6 9
```

[Bonus] Bubblesort

Beschreibung: `Arrays.sort()` uebernimmt das Sortieren automatisch. Aber wie funktioniert Sortieren eigentlich? Bubblesort ist einer der einfachsten Sortieralgorithmen: Benachbarte Elemente werden verglichen und bei Bedarf getauscht. Das wiederholt sich, bis die Liste vollstaendig sortiert ist.

Aufgabenstellung: Gegeben ist folgendes Array direkt im Code:

```
int[] zahlen = {5, 3, 8, 1, 9, 2, 7, 4};
```

Sortiere es mit dem Bubblesort-Algorithmus, ohne `Arrays.sort()` zu verwenden. Gib nach jedem vollstaendigen Durchlauf den aktuellen Zustand mit `Arrays.toString()` aus. Verwende zum Tauschen eine Hilfsvariable.

Ausgangsliste: [5, 3, 8, 1, 9, 2, 7, 4]
Durchlauf 1: [3, 5, 1, 8, 2, 7, 4, 9]
Durchlauf 2: [3, 1, 5, 2, 7, 4, 8, 9]
Durchlauf 3: [1, 3, 2, 5, 4, 7, 8, 9]
Durchlauf 4: [1, 2, 3, 4, 5, 7, 8, 9]
Durchlauf 5: [1, 2, 3, 4, 5, 7, 8, 9]
Durchlauf 6: [1, 2, 3, 4, 5, 7, 8, 9]
Durchlauf 7: [1, 2, 3, 4, 5, 7, 8, 9]
Sortiert: [1, 2, 3, 4, 5, 7, 8, 9]